

第一次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 1 · M1 编程本质

今日安排

知识讲解

Java 基础语法速览

`javac` / `java` · 数据类型 · 输入输出

AI实践 M1 编程本质

用 AI 写第一个程序，观察它的"假设"

大作业 大作业介绍 & 答疑

选题方向、提交规范、时间节点

当堂练习

练习 共 2 题，独立完成

- 练习 1: 读懂编译 & 执行链
- 练习 2: Hello World 变奏曲

目标：检验你对编译 / 解释执行的直觉理解

知识讲解

Java 语法 · AI4Coding M1 · 大作业介绍

Java 语法速览

编译 & 运行

```
javac Hello.java    # 编译 → Hello.class (字节码)
java Hello          # JVM 解释执行字节码
```

基本数据类型

类型	大小	示例
int	32bit	int x = 42;
double	64bit	double pi = 3.14;
boolean	1bit	boolean ok = true;
char	16bit	char c = 'A';
String	对象	String s = "hi";

标准输出 & 输入

```
System.out.println("带换行");
System.out.printf("格式化 %d%n", 42);

import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int n      = sc.nextInt();
String s   = sc.nextLine();
```

运算符陷阱

```
System.out.println(7 / 2);           // 3, 整数截断
System.out.println("x=" + 1 + 2);    // x=12, 拼接
System.out.println("x=" + (1 + 2));  // x=3, 先算术
```

Java vs C — 关键差异 (1/2)

String 是对象，不是字符数组

```
// C
char s[] = "hello";
strcmp(s, "hello") == 0 // 比较内容
```

```
// Java
String s = "hello";
s.equals("hello") // ✓ 比较内容
s == "hello"      // ✗ 比较引用地址!
```

String 是不可变对象； `s + "!"` 会创建新对象

数组有边界检查

```
int a[3] = {1, 2, 3};
a[5] = 99; // C: 未定义行为, 可能崩溃
printf("%d", sizeof(a)/sizeof(a[0])); // 手动求长度
```

```
int[] a = {1, 2, 3};
a[5] = 99; // ArrayIndexOutOfBoundsException
System.out.println(a.length); // ✓ 直接获取长度
```

没有指针，只有引用

```
int x = 42;
int *p = &x; // C: 可取地址、做指针运算
*p = 100;    // 解引用修改值
```

```
// Java: 对象变量是引用, 无法取地址
// 基本类型 (int/double/char) 直接存栈上, 无指针
// 对象 (String/数组) 存堆上, 变量持有引用
```

Java vs C — 关键差异 (2/2)

boolean 是独立类型

```
int flag = 0;
if (flag = 1) { ... } // C: 合法! 赋值返回1, 条件为真
```

```
boolean flag = false;
if (flag = true) { ... } // Java: 编译报错, boolean≠赋值表达式
if (flag == true) { ... } // ✔️ 或直接 if (flag)
```

Java 强类型: 条件表达式必须是 `boolean`, 彻底杜绝了 C 中"少写一个 `=`"的经典 Bug

自动内存管理 (GC)

	C	Java
分配	<code>malloc(size)</code>	<code>new ClassName()</code>
释放	<code>free(ptr)</code> (手动)	GC 自动回收
忘释放	内存泄漏	不存在此问题
悬空指针	可能发生	不存在

char 是 16 位 Unicode

```
char c1 = 'A'; // 65, 与 C 相同
char c2 = '中'; // ✔️ 合法! C 的 char 做不到
char c3 = '\u4e2d'; // '中' 的 Unicode 写法

// 注意: char 参与运算时提升为 int
System.out.println('A' + 1); // 66, 不是 "B"
```

Java 输入输出 — BufferedReader

为什么用 BufferedReader?

Scanner 解析方便但较慢;

BufferedReader 只读字符串, 速度快 10x 以上

	Scanner	BufferedReader
使用难度	简单	稍复杂
读取速度	慢	快
适用场景	日常练习	大量输入 / 竞赛

基本用法

```
import java.io.*;
```

```
BufferedReader br = new BufferedReader(  
    new InputStreamReader(System.in));
```

```
String line = br.readLine();          // 读一整行 (含空格)  
String[] parts = line.split(" ");    // 按空格切分为数组  
String[] parts2 = line.split(",");   // 按逗号切分
```

读多行

```
int n = Integer.parseInt(br.readLine().trim()); // 读行数  
for (int i = 0; i < n; i++) {  
    String row = br.readLine();  
    // 处理每行...  
}
```

Java 输入输出 — 类型转换

parseInt / parseDouble / parseLong

```
// String → 数值类型
int    n = Integer.parseInt("42");           // → 42
double d = Double.parseDouble("3.14");       // → 3.14
long   l = Long.parseLong("9999999999");     // → 大整数
```

```
// 结合 BufferedReader — 读单个整数
int n = Integer.parseInt(br.readLine().trim());
```

```
// 读一行多个整数 (如输入: 3 1 4 1 5)
String[] tokens = br.readLine().split(" ");
int[] arr = new int[tokens.length];
for (int i = 0; i < tokens.length; i++) {
    arr[i] = Integer.parseInt(tokens[i]);
}
```

数值 → String

```
String s1 = String.valueOf(42);              // "42"
String s2 = Integer.toString(42);            // "42"
String s3 = "" + 42;                         // "42" (拼接触发转换)
```

常见陷阱

```
// ❌ NumberFormatException: 字符串含空白字符
Integer.parseInt("42\n");
Integer.parseInt(" 42 ");
```

```
// ✅ 先 trim() 去除首尾空白
Integer.parseInt("42\n".trim());
Integer.parseInt(" 42 ".trim());
```

`br.readLine()` 不含换行符, 但可能有前后空格,
养成习惯: 数值解析前先 `.trim()`

AI4Coding 实践 — M1 编程本质

任务 用 AI 工具完成以下步骤，记录你的发现：

Step 1: 让 AI 写 Hello World

"用 Java 写一个 Hello World 程序"

- 代码能直接运行吗？和你写的有什么不同？

Step 2: 追问细节

"解释每一行代码的作用"
"如果我想输出我的名字，怎么改？"

- AI 的解释准确吗？有没有遗漏重要信息？

Step 3: 挑战 AI

"不使用 System.out.println，用另一种方式输出"

- 方案合理吗？能运行吗？

Step 4: 反思 — 填表分享

问题	你的观察
AI 做了哪些 隐含假设 ？	
AI 的代码是否完全正确？	
AI 没有告诉你 的重要事项？	
AI 改变了什么？没改变什么？	

核心结论

AI 改变的是**生产方式**
没有改变**计算模型**
`1/2 = 0`，永远成立

AI 时代，编程的本质没变

变了

- 生产方式：AI 参与代码生成
- 学习路径：可以先跑起来再理解
- 调试方式：AI 辅助定位错误
- 文档获取：对话代替搜索

没变的

- 程序 = 机器可执行的精确指令
- 计算机仍然只执行**机器码**
- `1 / 2 = 0`，无论谁写的代码
- **逻辑错误**需要人类发现和修复

冯诺依曼结构没有改变

输入 → 存储 → 执行 → 输出
AI 改变的是"谁在写指令"
没有改变"指令如何被执行"

你今天学到的核心

```
源代码 (人/AI 写)
    ↓ javac 编译
字节码 (.class)
    ↓ JVM 解释执行
机器指令 (CPU执行)
    ↓
输出结果
```

AI 可以帮你写源代码，
但理解这条链路，是你的责任。

大作业 — 项目一（必选）

Lambda 演算解释器

Lambda 演算是所有函数式语言的数学基础，也是理解"计算本质"的最小模型

输入 / 输出

输入：Lambda 演算源代码

```
(\x. \y. x) a b  
(\f. f f) (\f. f f)
```

输出：规约后的运行结果 (Normal Form)

```
a  
∞ (发散 / 超时)
```

核心挑战

1. **词法分析 & 语法解析** (Parser)
将源码解析为抽象语法树 (AST)
2. **α -变换**: 变量重命名, 避免自由变量被捕获
3. **β -规约**: 函数应用, 反复替换直到 Normal Form
4. **发散检测**: 识别无限规约 (循环/超时策略)

这是一个**真正的解释器**, 将让你理解:
所有高级语言的函数调用, 本质都是 β -规约

大作业 — 项目二（2 选 1）

选项 A: JVM 模拟器

输入: Java `.class` 字节码文件

输出: 模拟执行结果

模拟流程:

1. 解析 Class 文件: magic number / 常量池 / 字段表 / 方法表
2. 寻找并加载类: 模拟类路径查找
3. 生成 Class 对象: 构建类元信息结构
4. 定位目标方法: 找到字节码指令序列
5. 模拟执行: 操作数栈 + 局部变量表, 逐条解释字节码指令

让你从底层理解 `javac` 生成的字节码是如何被 JVM 解释运行的

选项 B: JS 模拟器

输入: JavaScript 源文件 (`.js`)

输出: 执行结果 (`console.log` 等输出)

核心流程:

JS 源码

↓ 词法分析 (Lexer)

Token 流

↓ 语法分析 (Parser)

抽象语法树 (AST)

↓ 树遍历解释执行 (Evaluator)

执行结果

与项目一的 Lambda 解释器同构:

都是"源码 → AST → 求值"的解释器架构

课后作业

必做

1. 完成今日练习（如未做完）
提交到 SEECODER，截止 48 小时内
2. 安装 AI 编程工具（二选一）
 - CodeBuddy（腾讯，推荐）
 - 通义灵码（阿里）
3. 运行你的第一个 Java 程序
用 AI 辅助，完成"Hello, 你的名字"

选做（加分）

- 尝试让 AI 解释 `public static void main(String[] args)` 的每个关键字
- 记录 AI 解释中你认为不够准确的地方
- 思考：同一个需求，用不同的 Prompt 问 AI，结果一样吗？

预习

下周内容：程序的结构、降低复杂度、算法建模

带着问题来：什么是"好的程序结构"？

当堂练习

当堂练习 · 独立完成 · 25 分钟

练习 1 — 读懂编译 & 执行链

15 min

下面是一段 Java 代码，请回答问题：

```
public class Calc {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 3;  
        System.out.println(x / y);  
        System.out.println((double) x / y);  
        System.out.println(x % y);  
    }  
}
```

① 程序输出是什么？ 写出每一行的结果。

② 画出执行链：

Calc.java → ____ → ____ → 屏幕输出

③ 下列说法哪个正确？（单选）

- A. Java 源码直接在 CPU 上运行
- B. javac 将源码编译为机器码
- C. java 命令运行的是字节码文件
- D. Java 和 C 的执行方式完全相同

④ 如果把 int x 改为 double x，哪行输出会变？

练习 2 — Hello World 变奏曲

10 min

下面这段代码能否成功编译并运行？如果不能，指出错误并修正。

版本 A

```
public class Hello {  
    public static void main(String[] args) {  
        String name = "世界";  
        System.out.println("Hello, " + name + "!");  
        System.out.println("2 + 3 = " + 2 + 3);  
        System.out.println("2 + 3 = " + (2 + 3));  
    }  
}
```

输出分别是什么？为什么第 2、3 行结果不同？

版本 B

```
public class hello {  
    public void Main(String[] args) {  
        system.out.println("Hello!")  
    }  
}
```

找出 所有错误（至少 4 处），并说明每处是什么类型的错误（编译期 / 运行期 / 逻辑）。

附录A Java 运行环境搭建

JDK 安装与环境搭建

1. 下载 JDK: <http://www.oracle.com/technetwork/java/javase/downloads/>

2. 环境变量设置

变量	值
JAVA_HOME	JDK 安装目录
Path	最前端加入 %JAVA_HOME%\bin;
CLASSPATH	.;%JAVA_HOME%\lib\dt.jar;%JAVA_HOME%\lib\tools.jar

3. 测试

```
java -version
javac -version
```

附录B AI4Coding 运行环境搭建

安装实验环境 — 1

- AI编程 IDE
 - VS Code 或者 IntelliJ IDEA
 - 配合 文心快码 或 通义灵码 插件

通义灵码 — VS Code 安装

- 官方网址: <https://tongyi.aliyun.com/lingma/>
- 文档: <https://help.aliyun.com/zh/lingma/support/faq>

安装步骤:

1. 在 VS Code 侧边导航点击「扩展」
2. 搜索 **TONGYI Lingma** , 点击安装
3. 重启 VS Code
4. 登录阿里云账号, 即可使用

通义灵码 — IntelliJ IDEA 安装

安装步骤：

1. 点击设置（`Ctrl+Alt+S`）
2. 在「插件」中搜索 `TONGYI Lingma`，点击安装
3. 重启 IntelliJ IDEA
4. 登录阿里云账号，即可使用

文心快码 — VS Code 安装（步骤1）

- 官方网址: <https://comate.baidu.com/zh>
- 文档: <https://comate.baidu.com/zh/readme>

安装 Baidu Comate:

1. 在 VS Code 侧边导航单击「扩展」
2. 在搜索框中输入 `Baidu Comate`
3. 找到 Baidu Comate 后单击 **Install** 安装

文心快码 — VS Code 安装（步骤2）

插件配置：

1. 安装后，VS Code 右下角弹出登录提示框，单击「去登录」
2. 唤起登录页面，支持百度、微信、微博、QQ 扫描登录
3. 登录后可免费使用基础功能
 - 高级功能每周免费使用 **300次**
 - 如需无限制使用可前往购买
4. 返回 VS Code 开始编码，即可体验各项功能

文心快码 — IntelliJ IDEA 安装（步骤1）

安装 Baidu Comate:

1. 点击 IntelliJ IDEA 界面中的 `File` → `Settings` → `Plugins`
2. 点击 **Marketplace** 选项卡
3. 在搜索框中输入插件名称 `comate`
4. 选择 Baidu Comate 插件，单击 **Install** 并重启

文心快码 — IntelliJ IDEA 安装（步骤2）

插件配置：

1. 插件安装重启后，右下角弹出登录提示框
2. 点击去登录链接，支持百度、微信、微博、QQ 扫描登录
3. 登录后可免费使用基础功能
 - 高级功能每周免费使用 **300次**
4. 返回 IDE 开始编码，即可体验 Comate 编码助手的各项功能

CodeBuddy — 安装（腾讯云AI代码助手）

<https://copilot.tencent.com/>

VS Code 安装

1. 打开 VS Code，进入 **Extensions** (`Ctrl+Shift+X`)
2. 搜索 **CodeBuddy**，找到腾讯云出品的插件
3. 点击 **Install**，安装完成后重启 VS Code
4. 左侧活动栏出现 CodeBuddy 图标，点击进入
5. 点击 **登录**，使用微信扫码或腾讯云账号授权

IntelliJ IDEA 安装

1. 打开 **Settings → Plugins → Marketplace**
2. 搜索 **CodeBuddy**，点击 Install
3. 重启 IDE 后，右侧工具栏出现 CodeBuddy 面板
4. 登录腾讯云账号即可使用

主要功能

功能	说明
代码补全	行级 / 块级实时补全
对话编程	侧边栏对话，理解项目上下文
代码解释	选中代码一键解释
单测生成	自动为函数生成单元测试
注释生成	补全函数/类的文档注释
代码优化	识别坏味道并给出建议

免费额度

- 代码补全：**无限次**
- 对话问答：每月 **500次** 免费
- 底层模型：混元代码大模型（腾讯自研）

附录C Git 基础

Git 是什么？

版本控制系统

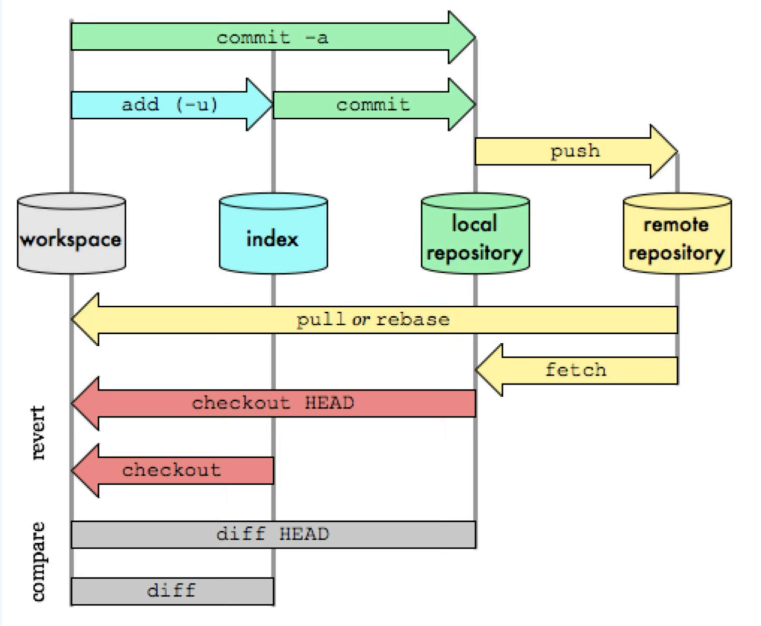
Git：分布式版本控制，记录每次变更，可随时回溯

为什么需要？

- 不再靠 作业_v1 / 作业_最终版 区分版本
- 多人协作合并代码不混乱
- AI 生成的代码可以逐步提交、随时回退

	文件备份	Git
历史	手动	自动
对比	难	<code>git diff</code>
协作	混乱	分支合并
回滚	找旧文件	<code>git checkout</code>

Git workflows 全景



- **Commit**：一次快照 + 说明信息
- **HEAD**：当前位置指针

Repository — 仓库的概念

本地仓库

```
git init          # 初始化新仓库
git status        # 查看状态
git log --oneline # 查看历史
```

仓库本质是 `.git/` 隐藏目录，存储所有历史数据。

仓库结构

```
my-project/
├── .git/      ← 仓库数据，勿手动修改
│   ├── HEAD
│   └── objects/ ← 所有提交内容
├── src/Hello.java
└── README.md
```

远程仓库

平台	用途
GitHub	开源社区，全球最大
GitLab	企业私有部署
SEECODER	本课程作业提交

关联远程仓库

```
# 查看已关联的远程
git remote -v

# 添加远程 (origin 是默认别名)
git remote add origin https://github.com/user/repo.git
```

git clone — 获取远程仓库

语法 & 常见用法

```
# 基本用法（自动创建同名文件夹）
git clone https://github.com/user/repo.git

# 克隆到指定目录名
git clone https://github.com/user/repo.git myproject

# 克隆指定分支
git clone -b main https://github.com/user/repo.git
```

clone 做了什么？

1. 下载远程仓库的**完整历史记录**
2. 自动设置 `origin` 指向远程地址
3. 检出默认分支（通常是 `main`）

实践：克隆课程仓库

```
git clone https://p-nju.seec.seecoder.cn/...
cd repo-name
ls
git log --oneline
```

注意

- `clone` 只需执行**一次**
- 已有本地仓库用 `git pull` 更新，勿重复 clone
- 地址来自仓库页面的 "Clone" 按钮

git push — 推送到远程仓库

完整工作流

```
git status           # 1. 查看状态
git add Hello.java   # 2. 加入暂存区
git commit -m "add Hello World" # 3. 提交本地
git push origin main # 4. 推送远程
```

```
# 首次推送, 设置上游 (之后可直接 git push)
git push -u origin main
```

push 做了什么?

将本地 `main` 分支的新 commit 上传到 `origin`

常见错误 & 解决

错误	原因	解决
rejected	远程有新提交	先 <code>git pull</code> , 再 push
Permission denied	未登录/无权限	检查账号配置
nothing to commit	没有新变更	先 add + commit

AI 辅助 Git

遇到 Git 报错, 把完整错误信息粘贴给 AI, 通常能快速定位原因

.gitignore — 为什么需要？ (1/2)

哪些文件不应提交？

类型	示例	原因
编译产物	<code>*.class</code> 、 <code>target/</code>	可重新生成
IDE 配置	<code>.idea/</code> 、 <code>*.iml</code>	每人环境不同
密钥配置	<code>.env</code> 、 <code>secret.key</code>	安全风险
系统文件	<code>.DS_Store</code>	无意义噪音

工作原理

`.gitignore` 放在仓库根目录，每行一条规则：

- `target/` — 忽略整个目录
- `*.class` — 忽略所有 `.class` 文件
- `!config.xml` — `!` 表示例外，不忽略

没有 .gitignore 的后果

Untracked files:

- `target/` # ← 几百个编译产物
- `.idea/` # ← IDE 私人配置
- `secret.key` # ← 危险！密钥泄露

后果：

- 仓库体积膨胀，clone 速度慢
- 他人 clone 后 IDE 配置冲突
- 密钥意外公开**（最严重）

已追踪的文件不受 `.gitignore` 影响，
需先 `git rm --cached <file>` 取消追踪

.gitignore — 推荐模板 (2/2)

Java / Maven 项目推荐模板

```
# Maven 构建输出
target/
# IDE 文件
.idea/
*.iml
.vscode/
# 编译产物
*.class
*.jar
# 系统文件
.DS_Store
Thumbs.db
# 敏感配置
# .env
```

快速生成

访问 <https://gitignore.io>

输入 Java, Maven, IntelliJ, VSCode

一键生成完整模板

取消追踪已提交文件

```
git rm --cached .DS_Store
git commit -m "stop tracking .DS_Store"
```

养成习惯：项目初始化时就创建 .gitignore，
避免事后补救的麻烦

附录D Maven 项目结构

Maven 是什么？

项目管理工具

Maven: Java 生态的构建 & 依赖管理工具
统一项目结构、自动下载第三方库、一键构建打包

解决的问题

没有 Maven	有 Maven
手动下载 jar 包	声明依赖自动下载
项目结构各不同	约定统一目录布局
构建命令复杂	<code>mvn package</code> 一键完成
版本冲突难排查	依赖树自动解析

安装验证

```
mvn -version
# Apache Maven 3.x.x
```

核心概念

- **POM** (Project Object Model): `pom.xml` 是项目的"身份证"
- **坐标**: `groupId` + `artifactId` + `version` 唯一标识一个库
- **仓库**: 本地缓存 `~/.m2/` + 中央仓库 + 私有仓库
- **生命周期**: `compile` → `test` → `package` → `install` → `deploy`

常用命令

```
mvn compile      # 编译源码
mvn test         # 运行测试
mvn package      # 打包为 jar/war
mvn clean        # 清理 target/ 目录
mvn dependency:tree # 查看依赖树
```

Maven 标准目录结构

```
my-project/
├── pom.xml                ← 项目配置文件
├── src/
│   ├── main/
│   │   ├── java/          ← 源代码
│   │   │   └── com/example/
│   │   │       └── App.java
│   │   └── resources/      ← 配置文件 (properties, xml)
│   └── test/
│       ├── java/          ← 测试代码
│       │   ├── com/example/
│       │   └── AppTest.java
│       └── resources/      ← 测试配置
└── target/                ← 构建输出 (自动生成, 勿提交 Git)
    ├── classes/
    └── my-project-1.0.jar
```

约定优于配置：只要遵循这个目录结构，Maven 不需要额外配置就能找到源码和测试。

关键规则

- `src/main/java/` — 生产代码，包名对应目录层级
- `src/test/java/` — 测试代码，包名与生产代码镜像
- `target/` — 不要提交到 Git，加入 `.gitignore`

.gitignore 示例

```
# Maven 构建输出
target/

# IDE 文件
.idea/
*.iml
.vscode/

# 编译产物
*.class
```

pom.xml — 项目配置文件

最小化 pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project>
  <modelVersion>4.0.0</modelVersion>

  <!-- 项目坐标 -->
  <groupId>com.example</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <!-- 依赖声明 -->
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

坐标三要素

字段	含义	示例
groupId	组织/公司域名倒写	com.example
artifactId	项目名称	my-project
version	版本号	1.0-SNAPSHOT

SNAPSHOT vs RELEASE

- 1.0-SNAPSHOT：开发中，可被覆盖更新
- 1.0：正式发布，不可变

搜索依赖坐标

<https://mvnrepository.com/>
搜索库名 → 复制 `<dependency>` 代码块 → 粘贴到 pom.xml